



Introduction to Algorithms

Homework 1

Recommended deadline - 2/11

1 Peaks Finding

Recall the logarithmic-time version of find-peak. In the recitation, we started to prove its correctness. In particular, we handled the first two splits in the if-else junctions, lines (2)-(7). Complete the proof by treating the last split, namely lines (8)-(10).

Result: returns a peak of $A_1 \dots A_n \in \mathbb{R}$

```
1  $i \leftarrow \lceil n/2 \rceil$ 
2 if  $A_i$  is a peak then
3   | return  $i$ 
4 end
5 else if  $A_{i-1} \leq A_i$  then
6   | return  $i + \text{find-peak}(A_{i+1} \dots A_n)$ 
7 end
8 else
9   | return  $\text{find-peak}(A_1, A_2, A_3 \dots A_{i-1})$ 
10 end
```

Algorithm 1: Improved algorithm

2 Peaks-Finding on the Cycle

Consider the following variation of the peaks-finding problem. We will define the peaks again to be the local maximas of given array A . However instead of thinking on position $1, 2 \dots n$ as coordinates on the line we will think about them as coordinates on the cycle. Namely A_1 is the right neighbor of A_n .

Show a series of arrays $\{A_n\}_{n=3}^{\infty}$ for which the recursive version of peak-find always fails. Explain in one line why the naive algorithm is still correct (You don't have to provide a formal correctness proof).

3 k -range peaks

In this section we define k -range peak to be all the items satisfy:

$$A_i = \max_{|i-j| \leq k} \{A_j\}$$

Write an algorithm that find a k -range peak in at most $k \log(n)$ up to constant factor independent on k and n . Prove correctness, and bound the running time.

4 2D-peaks

Let A be a $n \times n$ matrix. We will say that A_{ij} is a peak if:

$$A_{ij} \geq A_{i-1,j}, A_{i+1,j}, A_{i,j-1}, A_{i,j+1}$$

If the indices exceed the matrix frame, we define the inequality as satisfied (For example if $A_{1,1} \geq A_{1,2}, A_{2,1}$ then $A_{1,1}$ is a peak).

Concurrent examples. Consider the following matrices:

$$A = \begin{bmatrix} 1 & 0 \\ 0 & 0 \end{bmatrix} \quad B = \begin{bmatrix} 1 & 2 \\ 0 & 1 \end{bmatrix} \quad C = \begin{bmatrix} 1 & 0 & 0 \\ 1 & 2 & 0 \\ 0 & 0 & 0 \end{bmatrix}$$

The peaks of A are $A_{1,1} = 1$ and $A_{2,2} = 0$. B has only a single peak: $B_{1,2} = 2$. C has four peaks: $C_{1,1} = 1$, $C_{2,2} = 2$, $C_{3,3} = 0$, and $C_{1,3} = 0$.

Write an algorithm that find a peak in A . Solutions that runs in at most $cn \log n$ are sufficient. Prove correctness, and bound the running time.